

Control de tiempos y optimización paramétrica

1 Introducción

En este trabajo práctico implementará modelos de optimización matemática y algoritmos iterativos utilizando PYSCIPOPT. El objetivo central es aprender a gestionar límites de tiempo y resolver problemas de decisión jerárquicos donde la capacidad global de la planta está sujeta a un balance económico entre el beneficio operativo y el costo de infraestructura.

2 Descripción del problema

Una planta manufacturera debe determinar el plan de producción óptimo para un conjunto de n diseños de paneles acústicos de alta gama, con el objetivo de maximizar su beneficio neto total.

Sean $\mathcal{N} = \{1, \dots, n\}$ el conjunto de diseños y $\mathcal{M} = \{1, \dots, m\}$ el conjunto de recursos internos limitados disponibles en la planta (por ejemplo, horas de ensamblaje, volumen de resinas u horas de control de calidad). Cada diseño $j \in \mathcal{N}$ reporta un beneficio unitario c_j y requiere una cantidad $a_{ij} \in \mathbb{Z}$ del recurso $i \in \mathcal{M}$. La disponibilidad total de cada recurso i está acotada superiormente por un valor $b_i \in \mathbb{Z}_{\geq 0}$.

Adicionalmente, la producción de cada diseño j posee una cota superior de fabricación $u_j \in \mathbb{Z}_{\geq 0}$, motivada por limitaciones del mercado o restricciones de capacidad en la línea de montaje.

2.1 Capacidad global y costo de infraestructura

La operación de la planta está sujeta a una restricción global de capacidad impuesta por la gerencia a través de la *potencia eléctrica contratada* $\tau \in \mathbb{Z}_{\geq 0}$ (medida en kW). Cada unidad fabricada del diseño j consume $w_j \in \mathbb{Z}_{\geq 0}$ unidades de potencia.

El costo asociado a dicha infraestructura energética es no lineal y se modela mediante la función cuadrática:

$$\text{Costo}(\tau) = \beta\tau^2,$$

donde $\beta > 0$ representa el factor de deseconomía de escala.

Definiendo la cota superior teórica de potencia como $U = \sum_{j=1}^n w_j u_j$, el objetivo estratégico de la gerencia es encontrar el nivel óptimo de potencia contratada $\tau \in \{0, \dots, U\}$ que maximice el beneficio neto total:

$$\max_{\tau \in \{0, \dots, U\}} \left(\Pi(\tau) - \beta\tau^2 \right),$$

donde $\Pi(\tau)$ denota el beneficio operativo óptimo de producción, sujeto a que el consumo total de potencia no supere el límite τ .

3 Consigna

Antes de ejecutar los scripts sobre las instancias masivas provistas, se recomienda fuertemente que diseñe y testee su código utilizando una instancia reducida (por ejemplo, $n = 4$ y $m = 3$) para verificar la correcta formulación del modelo, la actualización de los parámetros y la lógica de los cortes de tiempo.

Lectura de datos

Para ambos ejercicios, las instancias deben ser cargadas desde un archivo externo mediante una función con la siguiente aridad, donde `path_instancia` representa la ruta al archivo de datos:

```
def leer_instancia(path_instancia: str):  
    ...
```

Esta función procesará el archivo correspondiente para extraer los parámetros enteros del problema ($n, m, c_j, b_i, a_{ij}, w_j, u_j$) y el parámetro de costo (β).

3.1 Etapa 1: modelo con límite de tiempo y capacidad fija

Implemente una función que reciba la ruta del archivo de instancia, el límite de tiempo y un valor fijo de capacidad:

```
def resolver_modelo(path_instancia, segundos, tau):  
    ...
```

Esta función cargará la instancia (utilizando `leer_instancia`) y construirá el modelo de programación lineal entera correspondiente, fijando la capacidad global en el valor entero τ dado.

Ejecute el solver con este modelo, limitando el tiempo de resolución mediante `model.setParam('limits')` y mida el tiempo real de ejecución con `time.monotonic()`.

```
import time  
  
inicio = time.monotonic()  
resolver_modelo(...)   
fin = time.monotonic()  
  
tiempo_utilizado = fin - inicio
```

3.2 Etapa 2: algoritmo iterativo con control global de tiempo

Implemente un procedimiento algorítmico iterativo para hallar la capacidad óptima que maximiza el beneficio neto, estructurado bajo la siguiente aridad:

```
def busqueda_tau(path_instancia, segundos):  
    ...
```

Donde `path_instancia` representa la ruta al archivo de datos y `segundos` corresponde al límite de tiempo global.

- **Exploración del espacio de búsqueda:** Queda a criterio de cada grupo el diseño de la estrategia para recorrer o explorar los distintos valores posibles de τ dentro del rango establecido, adaptándose al presupuesto de tiempo disponible.

- **Tiempo global:** El procedimiento debe respetar el límite de tiempo global indicado por `segundos`. Utilice `time.monotonic()` para medir el tiempo transcurrido y calcular el restante. No inicie una nueva iteración si el tiempo restante no es suficiente para finalizarla.
- **Actualización eficiente:** En cada iteración, actualice el modelo modificando únicamente el valor de τ , **sin reconstruirlo desde cero**. Modifique el lado derecho mediante `model.chgRhs()` y utilice `model.freeTransform()` cuando sea necesario. Esto permite al solver potencialmente reutilizar información obtenida en resoluciones anteriores, lo que puede resultar más eficiente.
- **Robustez:** Informe claramente si el solver no alcanza a obtener una primera solución factible.

Reporte y modalidad de entrega

En ambos ejercicios, el script debe reportar:

- la mejor solución factible encontrada, si existe;
- el valor de τ correspondiente a dicha solución, si existe;
- si la solución encontrada tiene garantía de optimalidad;
- tiempos de resolución: el solicitado y el efectivamente utilizado.

Formato de entrega: Se debe entregar el código fuente de la solución junto con un archivo Excel estructurado con 5 columnas bajo el formato: `{instancia: 4 datos del reporte}`.

3.3 Tests unitarios

Debe incluir un conjunto de pruebas utilizando `unittest` o `pytest`, validando:

- La correcta lectura de archivos válidos (provenientes del generador) y el manejo de excepciones ante archivos mal formados o inexistentes.
- Que una instancia chica ($n = 3, m = 2$) genere correctamente el modelo y devuelva una solución factible u óptima según corresponda.
- Que el control de tiempo global y por iteración se realice correctamente, evitando iniciar nuevas iteraciones si el tiempo restante es insuficiente.
- Que el manejo de estados del solver (`optimal`, `timelimit`, `infeasible`) sea robusto en ambas etapas, evaluando escenarios donde, por ejemplo, el tiempo se agote antes de hallar una primera solución factible.

Plazo límite de entrega: **7-oct-2026**.